

Infrastructure / SRE Intern Assignment

Project: Incident Management System

Candidate: Nida Farheen Khan

GitHub Link: <https://github.com/nidak7/mission-critical-ims-assignment-2026-04-30>

This PDF is the final submission artifact for the assignment and points to the public GitHub repository containing the full codebase, packaging, tests, and documentation.

Running Application Confirmation

The application is runnable both locally and through Docker Compose. The repository includes the backend, frontend dashboard, sample data scripts, tests, configuration, and packaging files needed for evaluator review.

Local run commands

```
python sample-data/reset_demo_state.py
python backend/run_server.py
python sample-data/send_scenario.py
```

Open:
`http://localhost:8000`

Docker command

```
docker compose up --build
```

GitHub / Packaging Confirmation

GitHub usage is mandatory for this assignment. The public repository contains the backend code, frontend/dashboard files, automated tests, sample data scripts, Docker Compose setup, README, configs/build scripts, and this final report PDF.

- Backend code
- Frontend/dashboard files
- Tests
- Sample data scripts
- Docker Compose
- README
- Configs/build scripts
- Final report PDF

Project Overview

The Incident Management System ingests operational signals, groups repeated signals from the same component, creates incidents, supports a workflow-driven incident lifecycle, requires RCA before closure, and calculates MTTR from RCA start and end time.

Architecture

```
Signal API
-> Rate Limiter
-> Async Queue
-> Debounce Processor
-> Incident Service

Incident / RCA -> SQLite
Raw Signals    -> JSONL
Dashboard      -> Backend APIs
```

Features Implemented

- Async signal ingestion

- Bounded queue / backpressure handling
- Debouncing for repeated signals
- Raw signal storage in JSONL
- Incident dashboard
- Severity sorting
- Workflow: OPEN -> INVESTIGATING -> RESOLVED -> CLOSED
- Mandatory RCA before close
- MTTR calculation
- Health endpoint
- Throughput logging
- Rate limiting
- Retry logic for DB writes
- Automated tests

Non-functional / Bonus Items

- Rate limiting to protect the ingestion API
- Bounded async queue to avoid crashes during bursts
- Retry logic for database writes
- Backend workflow validation
- Frontend prevention of invalid transitions
- Health endpoint for observability
- Throughput metrics every 5 seconds
- Clean error handling
- Docker packaging
- Automated tests

Testing Done

- Health endpoint test
- Signal ingestion test
- Invalid payload test
- Debouncing test
- Raw signal linking test
- Valid workflow transition test
- Invalid transition test
- RCA validation test
- MTTR calculation test

- Rate limiting test
- Dashboard state test

Test command

```
python -m unittest discover -s backend/tests
```

Demo Flow

- 1 Start app
- 2 Load sample data
- 3 Open dashboard
- 4 Select RDBMS_PRIMARY_01
- 5 Mark Investigating
- 6 Mark Resolved
- 7 Fill and save RCA
- 8 Close Incident
- 9 Verify MTTR and status

Known Limitations

- SQLite is used for assignment simplicity.
- JSONL is used instead of a production NoSQL store.
- The dashboard is a local lightweight implementation.
- A production version could use Kafka, Redis, PostgreSQL, MongoDB, and Prometheus/Grafana.

Conclusion

This submission meets the assignment requirements with a runnable Incident Management System, documented setup, automated tests, packaging through GitHub and Docker Compose, and a single PDF artifact for evaluation.